Assignment-13.3

Task 1: Refactoring – Removing Code Duplication

Objective

To eliminate repeated logic by extracting reusable functions.

Code:-

```python
# Legacy script with repeated logic
print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))
print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))
print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))
#Refactored the code by removing duplicate logic and create reusable functions.
#That calculates area and perimeter of a rectangle.
#Add proper docstrings and call the functions for the given values.
def calculate_area(length, width):
...

def calculate_perimeter(length, width):
    """Calculate the perimeter of a rectangle."""
    return 2 * (length + width)

# Call the functions for the given values
print("Area of Rectangle:", calculate_area(5, 10))
print("Perimeter of Rectangle:", calculate_perimeter(5, 10))
print("Area of Rectangle:", calculate_area(7, 12))
print("Perimeter of Rectangle:", calculate_perimeter(7, 12))
print("Area of Rectangle:", calculate_area(10, 15))
print("Perimeter of Rectangle:", calculate_perimeter(10, 15))
```

Output:-

```
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
```

Task 2: Refactoring – Optimizing Loops and Conditionals

Objective

To improve performance by replacing inefficient nested loops with optimized

structures.

Code:-

```
13.3(2).py > ...
#legacy code
#names = ["Alice", "Bob", "Charlie", "David"]
#search_names = ["Charlie", "Eve", "Bob"]
#for s in search_names:
#found = False
#if s == n:
#found = True
#if found:
#print(f"{s} is in the list")
#else:
#print(f"{s} is not in the list")
#Refactored the code to remove neseted loops.
#optimize the search operation using efficient data structures like sets.
#keep the output the same and make the code cleaner.
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]
names_set = set(names)
for s in search_names:
    if s in names_set:
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list")
```

Output:-

```
Charlie is in the list
Eve is not in the list
Bob is in the list
```

Task 3: Refactoring – Extracting Reusable Functions

Objective

To modularize code by extracting calculations into reusable functions.

Code:-

```python
#legacy code
#price = 250
#tax = price * 0.18
#total = price + tax
#print("Total Price:", total)
#price = 500
#tax = price * 0.18
#total = price + tax
#print("Total Price:", total)
#Refactor this Python code by extracting the repeated tax and total calculation into a reusable function called calculate_total(price).
#The function should calculate 18% tax and return the total price.
#Remove duplicated code and add proper docstrings.
def calculate_total(price):
    """
    Calculate the total price including 18% tax.

    Parameters:
    price (float): The original price of the item.

    Returns:
    float: The total price after adding tax.
    """
    tax = price * 0.18
    total = price + tax
    return total
# Calculate total price for different items
price1 = 250
total1 = calculate_total(price1)
print("Total Price for item 1:", total1)

price2 = 500
total2 = calculate_total(price2)
print("Total Price for item 2:", total2)
```

Output:-

```
PS C:\Users\gbsgu\OneDrive\Documents\Desktop\Assist
ding/13.3(3).py"
Total Price for item 1: 295.0
Total Price for item 2: 590.0
PS C:\Users\gbsgu\OneDrive\Documents\Desktop\Assist
```

Task 4: Refactoring – Replacing Hardcoded Values with Constants

Objective

To improve maintainability by replacing magic numbers with named constants.

Code:-

```
#print("Area of Circle:", 3.14159 * (7 ** 2))
#print("Circumference of Circle:", 2 * 3.14159 * 7)
#Refactor this Python code by replacing hardcoded values with named constants.
#Create constants for PI and RADIUS.
#Make the code cleaner and more maintainable while keeping the same output.
PI = 3.14159
RADIUS = 7
print("Area of Circle:", PI * (RADIUS ** 2))
print("Circumference of Circle:", 2 * PI * RADIUS)
```

Output:-

```
Area of Circle: 153.93791
Circumference of Circle: 43.98226
```

Task 5: Refactoring – Improving Variable Naming and Readability

Objective

To enhance readability using descriptive variable names and comments.

Task Description

Refactor a script with unclear variable names.

Code:-

```
#a = 10
#b = 20
#c = a * b / 2
#print(c)
#Refactor this Python code by improving variable names and readability.
#Replace unclear variables with descriptive names and add inline comments explaining the logic.
#Keep the output the same.
#Refactored code with improved variable names and added comments for clarity.
# Define the base and height of the triangle
base_length = 10
height_length = 20
# Calculate the area of the triangle using the formula: (base * height) / 2
triangle_area = (base_length * height_length) / 2
# Print the calculated area of the triangle
print(triangle_area)
```

Output:-

```
e/Documents/Desktop/Assisted coding/13.3(5).py"
100.0
```